

Additional Resource for 7.2D Task

Below is a guide to understanding the protocols in **Cisco Packet Tracer**.

1. IoT Protocols Overview

To support IoT use cases, a network requires a stack that handles everything from physical connectivity to high-level data exchange.

Communication Layers

- **Network/Link Layer:** Bluetooth (IEEE 802.15.1), Zigbee (IEEE 802.15.4), Wi-Fi (802.11), and LoRaWAN.
- **Transport Layer:** Primarily **UDP** (used by CoAP for speed/low overhead) and **TCP** (used by MQTT for reliability).

Application Protocols

Protocol	Best For	Characteristics
MQTT	Remote monitoring	Publish/Subscribe, extremely lightweight, requires a Broker.
CoAP	Battery-constrained devices	REST-based (GET/POST), runs over UDP, low overhead.
HTTP	High-bandwidth IoT	Heavy overhead, but easy to integrate with web services.
AMQP	Business messaging	High reliability, queuing, and security.

2. What is MQTT?

MQTT (Message Queuing Telemetry Transport) is a standard messaging protocol for the Internet of Things. It is designed as an extremely lightweight **publish/subscribe** messaging transport.

- **The Broker:** The central hub (**Laptop/PC**). It receives all messages and filters them to subscribers.
- **Publishers:** Devices that send data (e.g., a Temperature Sensor publishing to the topic home/temp).
- **Subscribers:** Devices that want data (e.g., a Fan that turns on when it receives a message from home/temp).
- **Topics:** The "address" of the data (e.g., home/living room/light).

Method 1:

Smart Home with MQTT using User Apps (MQTT client/broker)

To set up your IoT system in CPT, start by creating the physical topology: place a Home Gateway (DLC100) and configure its SSID and password, then add a laptop to act as the MQTT broker and connect it to the gateway while enabling the MQTT service. Next, add IoT devices like the light, fan, garage door, and humidifier, switch their network adapters to wireless, and connect them to the same SSID. After that, program the devices such as the light using a Python MQTT client script that connects to the broker's IP, subscribes to a topic (e.g., "home/light"), and controls the device based on received messages. Finally, test the setup by running the script, using the laptop's MQTT client to publish commands (like "ON"), and verifying that the device responds, you can then extend the same logic to other devices by assigning different topics and payloads.

Method 2:

An alternative way to build your IoT system is by using an **SBC (Single Board Controller)** instead of relying only on the laptop as the broker. In this approach, the SBC can act as the central hub where you run the MQTT broker as well as write both publisher and subscriber programs. First, configure the SBC to connect to the Home Gateway network, then enable or install the MQTT service on it. After that, create simple scripts on the SBC: one acting as a *publisher* (sending commands like ON/OFF to topics such as *home/light*) and another as a *subscriber* (listening to those topics and controlling devices accordingly). Finally, connect your IoT devices to the same network and ensure they subscribe to the correct topics, so they can respond to messages sent from the SBC, effectively making it a compact all-in-one controller for your smart home system.

Video Links:

<https://www.youtube.com/playlist?list=PLS1QulWo1RIbQWue3zzwEuV2pSASiDxvv>

<https://youtu.be/IAR4GuAaMGg>